

LLM API 연동과 Tool Use 실습

AI Agent 시스템 구현

OpenAI / Claude API

Function Calling

금융 Tool 구현

Prompt Engineering

Hunter Shin [hunter.shin@wyhil.com]

01. 학습목표

01

Claude / OpenAI API 직접 구현

Function Calling 구조를 이해하고 금융 도구를 에이전트에 연결할 수 있다

02

금융 Tool 4종 구현

대출 계산기 · 뉴스 검색 · 환율 조회 · 신용점수 조회
도구를 직접 작성한다

03

ReAct 에이전트 완성

Thought→Action→Observation 루프가 작동하는
금융 에이전트를 완성한다

04

Prompt Engineering 적용

System Prompt · Few-shot · Role Assignment로
에이전트 행동을 제어한다

사전 준비 체크리스트

- ✓ pip install anthropic openai langchain langchain-community
- ✓ .env 파일에 ANTHROPIC_API_KEY / OPENAI_API_KEY 설정
- ✓ Python 3.11 이상 환경 확인 (3.12 권장)

02. OpenAI API 기본 구조 — messages.create()

model

gpt-5.4-mini

사용할 LLM 모델 선택

messages

[{"role": "user", "content": "..."}]

대화 히스토리 배열

tools

[{name, description, ...}]

에이전트가 사용할 도구 목록

stop_reason 두 가지 상태

end_turn

최종 응답 완성 → 텍스트 추출 후 반환

tool_use

도구 호출 필요 → 실행 후 결과 전달

```
from openai import OpenAI
import os

client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# TOOL목록
FINANCIAL_TOOLS = [ ]

response = client.messages.create(
    model = 'gpt-5.4-mini',
    system = '당신은 금융 전문 AI입니다.',
    tools = FINANCIAL_TOOLS, # Tool 목록
    input = [
        {
            'role': 'user',
            'content': '3억원 연 4.5% 20년',
        }
    ]
)

# 결과 확인
print(response.output)
```

03. Tool 정의 — JSON Schema 설계가 품질을 결정한다

✕ 나쁜 예

Description이 불명확하면 Claude가 언제 이 도구를 써야 할지 판단하지 못합니다.

```
{
  "name": "calc",
  "description": "계산",
  "input_schema": {
    "properties": {
      "a": {"type": "number"},
      "b": {"type": "number"}
    }
  }
}
```

✓ 좋은 예

사용 시점 · 입력 형식 · 단위를 명확히 기술하면 도구 선택 정확도가 크게 향상됩니다.

```
{
  "name": "calculate_loan_payment",
  "description": "원리금 균등 분할 방식으로 월 상환액을 계산합니다.  
대출 금액·금리·기간 계산 시 반드시 이 도구를 사용하세요.",
  "input_schema": {
    "properties": {
      "principal": {"type": "number",
        "description": "대출 원금(원). 3억 → 300000000"},
      "annual_rate": {"type": "number",
        "description": "연이율(%). 4.5% → 4.5"},
      "months": {"type": "integer",
        "description": "기간(개월). 20년 → 240"}
    },
    "required": ["principal", "annual_rate", "months"]
  }
}
```

04. 금융 Tool 4종 완전 구현



대출 월 상환액 계산

calculate_loan_payment

원리금 균등 분할 공식

$$P \times r(1+r)^n / ((1+r)^n - 1)$$

```
r = annual_rate/100/12
```

```
pmt = P*r*(1+r)**n / ((1+r)**n-1)
```

→ {월상환액, 총이자, 총상환액}



금융 뉴스 검색

search_financial_news

키워드로 최신 뉴스 검색

(실제: 네이버·Bloomberg API)

```
query: str, limit: int=3
```

→ {제목, 요약, 출처, 날짜}

뉴스 3~5건 반환



환율 조회

get_exchange_rate

USD/KRW, EUR/KRW 등

실시간 환율 조회

```
currency_pair: str
```

→ {환율, 전일대비, 등락률}

지원: USD/EUR/JPY/CNY



고객 신용점수 조회

check_credit_score

내부 CRM / KCB·NICE

신용정보 API 연동

```
customer_id: str
```

→ {신용점수, 등급, 연체여부}

PII 마스킹 필수!

05. ReAct 루프 — 핵심 코드 구조



```
def run_financial_agent(query, max_turns=8):  
    messages = [{'role': 'user', 'content': query}]  
  
    for turn in range(max_turns):  
        response = client.messages.create(  
            model='claude-sonnet-4-6', system=SYSTEM_PROMPT,  
            tools=FINANCIAL_TOOLS, messages=messages  
        )  
  
        if response.stop_reason == 'end_turn':  
            return response.content[0].text # 완료  
  
        # Tool 호출 처리  
        tool_results = []  
        for block in response.content:  
            if block.type == 'tool_use':  
                result = execute_tool(block.name, block.input)  
                tool_results.append({  
                    'type': 'tool_result', 'tool_use_id': block.id,  
                    'content': json.dumps(result)  
                })  
        messages.append({'role': 'assistant', 'content': response.content})  
        messages.append({'role': 'user', 'content': tool_results})  
  
    return '최대 턴 초과' # max_turns 안전장치
```

핵심 설계 포인트

max_turns 필수

무한 루프 방지 — 통상 8~15 설정

messages 누적

대화 히스토리 직접 관리 (LLM 자체 메모리 없음)

tool_result 구조

tool_use_id 일치 필수 — 다르면 오류 발생

execute_tool 안전

try-except 래핑 → dict 반환 (예외 전파 금지)

06. 실습 — 금리 계산기 + 뉴스 검색 에이전트 실행

TC-01

Q: 3억원을 연 4.5%로 20년 대출 시 월 상환액은?

도구: `calculate_loan_payment`

A: 월 1,897,236원 / 총이자 155,336,640원

3턴

TC-02

Q: 위 계산 결과와 최신 주담대 금리 동향 2건을 함께 알려줘

도구: `calculate_loan_payment` + `search_financial_news`

A: 복합 답변 (수치 + 뉴스 요약)

4턴

TC-03

Q: 고객 C001의 신용점수 확인 + USD/KRW 환율

도구: `check_credit_score` + `get_exchange_rate`

A: 820점 / 1등급 + 현재 환율 1,340.5원

3턴

[보안] 고객 ID (C001) 등 개인 식별 정보는 실제 환경에서 PII 마스킹 후 LLM에 전달하세요

07. Prompt Engineering — 에이전트 행동 제어 3대 기법

1 System Prompt

에이전트의 역할·원칙·형식·제약을 정의합니다.
명확할수록 일관된 응답 품질을 얻습니다.



```
system = ""당신은 금융회사 여신심사 AI입니다.
```

```
[필수 원칙]
```

- 계산은 반드시 도구를 사용하세요. 직접 계산 금지.
- 규정에 없는 내용은 추측하지 마세요.
- 금액은 쉼표 포함 원화로: 1,897,236원

```
[제약]
```

- 최종 대출 결정은 심사역이 수행합니다."

2 Few-shot 예시

원하는 입출력 형식을 예시로 보여줍니다.
복잡한 포맷이나 금융 표현에 특히 효과적입니다.



```
# 메시지 배열에 예시를 직접 포함
```

```
messages = [  
  {"role": "user", "content": "DTI가 뭔가요?"},  
  {"role": "assistant", "content": "DTI(총부채상환비율)는  
연소득 대비 연간 부채 원리금 비율입니다.\n[근거: 내부규정 제4조 ①]},  
  {"role": "user", "content": user_query}  
]
```

3 Role Assignment

구체적인 직책과 경력을 부여합니다.
역할이 명확할수록 전문 어투·판단 기준이 향상
됩니다.



```
# 모호한 역할 (나쁨)
```

```
system = '당신은 AI입니다.'
```

```
# 구체적 역할 (좋음)
```

```
system = ""당신은 KCB 공인 신용 분석가이며  
15년 경력의 여신 심사 전문가입니다.  
DTI·LTV 계산, 규정 해석, 리스크 평가를  
전문으로 합니다."
```


08. System Prompt 4대 구성 요소 — 금융 에이전트 설계 기준

역할 (Role)

직책.전문성.경력을
구체적으로 부여



당신은 금융회사의
여신심사 보조 AI입니다.

원칙 (Rules)

반드시 해야 할 것
절대 하지 말아야 할 것



계산은 도구 사용.
추측 금지. 근거 조항 명시.

형식 (Format)

응답 형식.단위.
표 구조 지정



금액: 십표 원화
결과: 표 형식 정리

제약 (Limits)

역할 범위 한정
보안.컴플라이언스



투자 보장 발언 금지
최종 결정은 심사역

완성 예시

system = '당신은 금융회사 여신심사 AI입니다. 계산은 반드시 도구를 사용하세요. 금액은 십표 포함 원화로 표시. 최종 결정은 심사역이 수행합니다.'

09. 실습 과제 — DTI Tool 추가 + 복합 에이전트 완성

3단계 과제

STEP 1	calculate_dti 함수 구현 연소득·기존부채·신규상환액 입력 → DTI(%) 계산 통과 여부(40% 이하) 반환
STEP 2	Tool 스키마 + 맵 추가 FINANCIAL_TOOLS에 스키마 추가 TOOL_MAP에 함수 연결
STEP 3	복합 질의로 에이전트 실행 연소득 6천만, 대출 3억, 기존부채 50만원 → 월 상환액 + DTI 계산 + 신용점수 조회

평가 루브릭

항목	A (우수)	B (보통)	C (미흡)
API 연동	오류 없이 실행, 두 도구 정상	연결 성공, 일부 미흡	API 키 오류
Tool 정의	스키마 완전, 예외 처리 포함	대부분 올바름	스키마 오류
에이전트 응답	정확한 결과 + 자연스러운 한국어	결과 맞으나 형식 미흡	오류 발생
코드 가독성	함수 분리, 주석 충실	핵심 주석 있음	주석 없음

10. MODULE 2 핵심 정리



API 구조

messages.create() + tools + stop_reason 3가지가 핵심



Tool 정의

description 품질이 도구 선택 정확도를 결정한다



ReAct 루프

max_turns 필수 · messages 직접 누적 · dict 반환 원칙



Prompt 설계

역할·원칙·형식·제약 4요소로 에이전트 행동을 제어한다

수고하셨습니다.

(주)시대의영웅은 AI/AX 분야의 컨설턴트 및 개발자로 이뤄진 전문가 집단입니다.
AI 개발에 필요한 경험과 기술을 축적하여 다양한 개발을 하고 있습니다.

추가적인 문의사항이 있으시거나 상담이 필요하신 경우 아래의 주소로 방문하시거나
전화, 이메일 등으로 연락주시기 바랍니다.

서울시 강서구 마곡중앙로 152, 두산더랜드타워 B동 701/701호

02 2658 2990

contact@wyhil.com

A01. Claude API 기본 구조 — messages.create()

model

claude-sonnet-4-6

사용할 LLM 모델 선택

messages

[{"role": "user", "content": "..."}]

대화 히스토리 배열

tools

[{name, description, ...}]

에이전트가 사용할 도구 목록

stop_reason 두 가지 상태

end_turn

최종 응답 완성 → 텍스트 추출 후 반환

tool_use

도구 호출 필요 → 실행 후 결과 전달

```
import anthropic

client = anthropic.Anthropic()

response = client.messages.create(
    model = 'claude-sonnet-4-6',
    max_tokens = 1024,
    system = '당신은 금융 전문 AI입니다.',
    tools = FINANCIAL_TOOLS, # Tool 목록
    messages = [
        {
            'role': 'user',
            'content': '3억원 연 4.5% 20년',
        }
    ]
)

# 결과 확인
print(response.stop_reason) # tool_use
print(response.content)    # ToolUseBlock
```